

On designing interval operators in CER

Fernando Riveros ✉

Pontificia Universidad Católica de Chile

Cristian Riveros ✉

Pontificia Universidad Católica de Chile

Abstract

These notes contain the definition and implementation with complex event automata of interval operators in CER.

2012 ACM Subject Classification Theory of computation → Database theory

Keywords and phrases Streams, query evaluation, enumeration algorithms.

1 Preliminaries

Sets and intervals. Given a set A , we denote by 2^A the *powerset* of A . We denote by $\mathbb{N}$ the natural numbers. Given $n, m \in \mathbb{N}$ with $n \leq m$, we denote by $[n]$ the set $\{1, \dots, n\}$ and by $[n..m]$ the interval $\{n, n+1, \dots, m\}$ over $\mathbb{N}$.

Events and streams. We fix a set $\mathbf{T}$ of *event types*, a set $\mathbf{A}$ of *attributes names*, and a set $\mathbf{D}$ of *data values* (e.g., integers, floats, strings). An *event* e is a partial mapping $e: \mathbf{A} \rightarrow \mathbf{D}$ that maps attributes names in $\mathbf{A}$ to data values in $\mathbf{D}$. We denote $\text{att}(e)$ the domain of e , called the attributes of e , and we assume that $\text{att}(e)$ is finite. We denote by $e(A)$ the data value of the attribute $A \in \mathbf{A}$ whenever $A \in \text{att}(e)$. Further, each event e has a type in $\mathbf{T}$ denoted by $\text{type}(e)$. We write $\mathbf{E}$ to denote the set of all events over event types $\mathbf{T}$, attributes names $\mathbf{A}$, and data values $\mathbf{D}$.

A *stream* is an (arbitrary long) sequence $\bar{S} = e_1 e_2 \dots e_n$ of events where $|\bar{S}| = n$ is the length of the stream.

Complex events. Fix a finite set $\mathbf{X}$ of variables and assume that $\mathbf{T} \subseteq \mathbf{X}$, where $\mathbf{T}$ is the set of event types as defined earlier, this is to say that all event types are a variable. Let $\bar{S}$ be a stream of length n . A complex event of $\bar{S}$ is a triple (i, j, μ) where $i, j \in [n]$, $i \leq j$, and $\mu: \mathbf{X} \rightarrow 2^{[i..j]}$ is a function with finite domain. Intuitively, i and j marks the beginning and end of the interval where the complex event happens, and μ stores the events in the interval $[i..j]$ that fired the complex event. In the following, we will usually use C to denote a complex event (i, j, μ) of $\bar{S}$ and omit $\bar{S}$ if the stream is clear from the context. We will use $\text{interval}(C)$, $\text{start}(C)$, and $\text{end}(C)$ to denote the interval $[i..j]$, the start i , and the end j of C , respectively.

Further, by some abuse of notation we will also use $C(X)$ for $X \in \mathbf{X}$ to denote the set $\mu(X)$ of C .

The following operations on complex events will be useful throughout the paper. We define the union of complex events C_1 and C_2 , denoted by $C_1 \cup C_2$, as the complex event C' such that $\text{start}(C') = \min\{\text{start}(C_1), \text{start}(C_2)\}$, $\text{end}(C') = \max\{\text{end}(C_1), \text{end}(C_2)\}$, and $C'(X) = C_1(X) \cup C_2(X)$ for every $X \in \mathbf{X}$. Further, we define the *projection over L* of a complex event C , denoted by $\pi_L(C)$, as the complex event C' such that $\text{interval}(C') = \text{interval}(C)$ and $C'(X) = C(X)$ whenever $X \in L$, and $C'(X) = \emptyset$, otherwise. Finally, we denote by $(i, j, \mu_\emptyset)$ the complex event with trivial mapping $\mu_\emptyset$ such that $\mu_\emptyset(X) = \emptyset$ for every $X \in \mathbf{X}$.

Predicate of events. A *predicate* is a possibly infinite set $\mathbf{P}$ of events. We say that an event e satisfies predicate P , denoted $e \models P$, if, and only if, $e \in P$. We generalize this notation

$$\begin{aligned}
\llbracket R \rrbracket(\bar{S}) &= \{ (i, i, \mu) \mid \text{type}(e_i) = R \wedge \mu(R) = \{i\} \wedge \forall X \neq R. \mu(X) = \emptyset \} \\
\llbracket \varphi \text{ AS } X \rrbracket(\bar{S}) &= \{ C \mid \exists C' \in \llbracket \varphi \rrbracket(\bar{S}). \text{interval}(C) = \text{interval}(C') \wedge C(X) = \bigcup_Y C'(Y) \\
&\quad \wedge \forall Z \neq X. C(Z) = C'(Z) \} \\
\llbracket \varphi \text{ FILTER } X[P] \rrbracket(\bar{S}) &= \{ C \mid C \in \llbracket \varphi \rrbracket(\bar{S}) \wedge C(X) \models P \} \\
\llbracket \varphi_1 \text{ OR } \varphi_2 \rrbracket(\bar{S}) &= \llbracket \varphi_1 \rrbracket(\bar{S}) \cup \llbracket \varphi_2 \rrbracket(\bar{S}) \\
\llbracket \varphi_1 \text{ AND } \varphi_2 \rrbracket(\bar{S}) &= \llbracket \varphi_1 \rrbracket(\bar{S}) \cap \llbracket \varphi_2 \rrbracket(\bar{S}) \\
\llbracket \varphi_1; \varphi_2 \rrbracket(\bar{S}) &= \{ C_1 \cup C_2 \mid C_1 \in \llbracket \varphi_1 \rrbracket(\bar{S}) \wedge C_2 \in \llbracket \varphi_2 \rrbracket(\bar{S}) \wedge \text{end}(C_1) < \text{start}(C_2) \} \\
\llbracket \varphi_1 : \varphi_2 \rrbracket(\bar{S}) &= \{ C_1 \cup C_2 \mid C_1 \in \llbracket \varphi_1 \rrbracket(\bar{S}) \wedge C_2 \in \llbracket \varphi_2 \rrbracket(\bar{S}) \wedge \text{end}(C_1) + 1 = \text{start}(C_2) \} \\
\llbracket \varphi^+ \rrbracket(\bar{S}) &= \llbracket \varphi \rrbracket(\bar{S}) \cup \llbracket \varphi; \varphi^+ \rrbracket(\bar{S}) \\
\llbracket \varphi^\oplus \rrbracket(\bar{S}) &= \llbracket \varphi \rrbracket(\bar{S}) \cup \llbracket \varphi : \varphi^\oplus \rrbracket(\bar{S}) \\
\llbracket \pi_L(\varphi) \rrbracket(\bar{S}) &= \{ \pi_L(C) \mid C \in \llbracket \varphi \rrbracket(\bar{S}) \}
\end{aligned}$$

■ **Figure 1** Figure 1: The semantics of CEL formulas defined over a stream $\bar{S} = e_1 e_2 \dots e_n$ where each e_i is an event.

from events to a set of events E such that $E \models P$ if, and only if, $e \models P$ for every $e \in E$. We assume a fixed set of predicates $\mathbf{P}$. Further, we assume that there is a basic set of predicates $\mathbf{P}_{\text{basic}} \subseteq \mathbf{P}$ and $\mathbf{P}$ is the closure of $\mathbf{P}_{\text{basic}}$ under intersection and negation (i.e., $P_1 \cap P_2 \in \mathbf{P}$ and $\mathbf{E} \setminus P \in \mathbf{P}$ for every $P, P_1, P_2 \in \mathbf{P}$) where $\mathbf{E}$ is a predicate in $\mathbf{P}$, that we usually denote by true.

Complex event logic. In this work, we use the Complex Event Logic (CEL) introduced in [21] and implemented in CORE [11] as our basic query language for CER. The syntax of a CEL formula φ is given by the grammar:

$$\varphi := R \mid \varphi \text{ AS } X \mid \varphi \text{ FILTER } X[P] \mid \varphi \text{ OR } \varphi \mid \varphi \text{ AND } \varphi \mid \varphi; \varphi \mid \varphi : \varphi \mid \varphi^+ \mid \varphi^\oplus \mid \pi_L(\varphi)$$

where $R \in \mathbf{T}$ is an event type, $X \in \mathbf{X}$ is a variable, $P \in \mathbf{P}$ is a predicate, and $L \subseteq \mathbf{X}$ is a set of variables. We define the semantics of a CEL formula φ over a stream $\bar{S}$, recursively, as a set of complex events over $\bar{S}$. In Figure 1, we define the semantics of each CEL operator like in [11, 21].

Cristian: TODO: agregar las citas con bibtex.

Complex Event Automata with ϵ -transitions. A *Complex Event Automata* (CEA) (with ϵ -transitions) is a tuple $\mathcal{A} = (Q, \mathbf{P}, \mathbf{X}, \Delta, q_0, F)$ where Q is a finite set of states, $\mathbf{P}$ is the set of predicates, $\mathbf{X}$ is a finite set of variables,

$$\Delta \subseteq Q \times (\mathbf{P} \times 2^{\mathbf{X}} \cup \{\epsilon\}) \times Q$$

is a finite relation (called the transition relation), $q_0 \in Q$ is the initial state, and F is the set of final states. A transition of the form (p, P, L, q) is a *reading transition* and a transition of the form (p, ϵ, q) is an ϵ -transition. As usual, the automaton can change its state without consuming any event of the stream by using an ϵ -transition.

Let $\bar{S} = e_1 e_2 \dots e_n$ be a stream. A configuration of $\mathcal{A}$ over $\bar{S}$ is a pair $(q, i) \in Q \times [n]$. A run ρ of $\mathcal{A}$ over the stream $\bar{S} = e_1 e_2 \dots e_n$ from position i to j with $i, j \in [n]$ and $i \leq j$ is a sequence of configurations and transitions of the form:

$$\rho := (p_1, i_1) \xrightarrow{t_1} (p_2, i_2) \xrightarrow{t_2} \dots \xrightarrow{t_k} (p_{k+1}, i_{k+1})$$

such that $t_\ell \in \mathbf{P} \times 2^{\mathbf{X}} \cup \{\epsilon\}$ for every $\ell \in [k]$, $p_1 = q_0$, $i_1 = i$, $i_{k+1} = j + 1$, and, for every $\ell \in [k]$:

- if $t_\ell = (P, L)$, then $i_{\ell+1} = i_\ell + 1$, $(p_\ell, P, L, p_{\ell+1}) \in \Delta$ and $e_{i_\ell} \models P$;
- if $t_\ell = \epsilon$, then $i_{\ell+1} = i_\ell$ and $(p_\ell, \epsilon, p_{\ell+1}) \in \Delta$.

We say that the run is accepting if $p_{k+1} \in F$. For $t \in (\mathbf{P} \times 2^{\mathbf{X}} \cup \{\epsilon\})$ and $\ell \in [n]$ we define the complex event $\text{ce}(t, \ell)$ of $\bar{S}$ such that $\text{ce}(t, \ell) = (\ell, \ell, \{X \mapsto \{\ell\} \mid X \in L\})$ if $t = (P, L)$ and $(\ell, \ell, \emptyset)$, otherwise. Then a run ρ from positions i to j like above defines the complex event C_ρ such that:

$$C_\rho := \text{ce}(t_1, i_1) \cup \text{ce}(t_2, i_2) \cup \dots \cup \text{ce}(t_k, i_k)$$

Note that when $t_\ell = \epsilon$, then the transition captures no event, but when $t_\ell = (P, L)$ the transition captures the event e_{i_ℓ} in each variable $X \in L$. We define the set of all complex events of $\mathcal{A}$ over $\bar{S}$ as:

$$\llbracket \mathcal{A} \rrbracket(\bar{S}) = \{C_\rho \mid \rho \text{ is an accepting run of } \mathcal{A} \text{ over } \bar{S}\}$$

2 Overlap operator

In this section we introduce an extension to the semantics of CEL, namely we introduce a new operator using *Allen interval algebra overlap*. We then extend the formal computational model for evaluating CEL formulas and prove its correctness.

We present then the *overlap operator* for CEL as with the following definition:

$$\begin{aligned} \llbracket \varphi_1 :o \varphi_2 \rrbracket(\bar{S}) &= \{C_1 \cup C_2 \mid C_1 \in \llbracket \varphi_1 \rrbracket(\bar{S}) \wedge C_2 \in \llbracket \varphi_2 \rrbracket(\bar{S}) \\ &\quad \wedge \text{start}(C_1) \leq \text{start}(C_2) \leq \text{end}(C_1) \leq \text{end}(C_2)\} \end{aligned}$$

► **Example 1.** Suppose we have sensors in a park monitoring humidity (H) and temperature (T). Consider a stream $\bar{S} = e_1 e_2 e_3 e_4$ where the event types are $\text{type}(e_1) = H$, $\text{type}(e_2) = T$, $\text{type}(e_3) = H$, and $\text{type}(e_4) = T$.

Let $\varphi_1 = H; H$ be a formula detecting a non contiguous sequence of humidity readings, and $\varphi_2 = T; T$ be a formula detecting a non contiguous sequence of temperature readings.

- The set $\llbracket \varphi_1 \rrbracket(\bar{S})$ contains a complex event C_1 representing the humidity sequence at positions 1 and 3, with $\text{interval}(C_1) = [1..3]$. Thus, $\text{start}(C_1) = 1$ and $\text{end}(C_1) = 3$.
- The set $\llbracket \varphi_2 \rrbracket(\bar{S})$ contains a complex event C_2 representing the temperature sequence at positions 2 and 4, with $\text{interval}(C_2) = [2..4]$. Thus, $\text{start}(C_2) = 2$ and $\text{end}(C_2) = 4$.

Since the overlap condition $\text{start}(C_1) \leq \text{start}(C_2) \leq \text{end}(C_1) \leq \text{end}(C_2)$ holds (i.e., $1 \leq 2 \leq 3 \leq 4$), the complex event $C = C_1 \cup C_2$ is in the result of $\llbracket \varphi_1 :o \varphi_2 \rrbracket(\bar{S})$. This resulting complex event captures the period where a humidity rise was followed by a temperature rise that continued after the humidity stabilized.

Cristian: Buen ejemplo!

Cristian: Aquí habla de la subida de humedad y de temperatura, pero antes no se menciona nada sobre φ_1 y φ_2 . Esta bien?

We denote by $\text{CEL} \cup \{ :o \}$ the *extension of CEL* with the overlap operation. We can prove the following result:

► **Theorem 2.** For every $\text{CEL} \cup \{ :o \}$ formula φ there exists a CEA $\mathcal{A}_\varphi$ such that $\llbracket \varphi \rrbracket(\bar{S}) = \llbracket \mathcal{A}_\varphi \rrbracket(\bar{S})$ for every stream $\bar{S}$.

Cristian: Cambie la presentación del teorema.

Proof. We already know that for every (standard) CEL operator we can construct an equivalent CEA. So, we concentrate next in the construction of a CEA for the overlap operator. Let φ_1 and φ_2 be formulas in CEL extended with the overlap operator. We show by induction that there exists a CEA $\mathcal{A}_{\cdot:o}$ such that:

$$\llbracket \varphi_1 :o \varphi_2 \rrbracket(\bar{S}) = \llbracket \mathcal{A}_{\cdot:o} \rrbracket(\bar{S})$$

Assume then that there exist CEAs that satisfy the previous property for φ_1 and φ_2 , called $\mathcal{A}_{\varphi_1} = (Q_1, \mathbf{P}_1, \mathbf{X}_1, \Delta_1, q_0, F_1)$ and $\mathcal{A}_{\varphi_2} = (Q_2, \mathbf{P}_2, \mathbf{X}_2, \Delta_2, p_0, F_2)$, respectively. Then the construction for the overlap operator is as follows. Let $\mathcal{A}_{\cdot:o}$ be a CEA where $\mathcal{A}_{\cdot:o} = (Q_{\cdot:o}, \mathbf{P}_{\cdot:o}, \mathbf{X}_{\cdot:o}, \Delta_{\cdot:o}, q_0, F_2)$:

$$\begin{aligned} Q_{\cdot:o} &= Q_1 \uplus (Q_1 \times Q_2 \times \{0, 1\}) \uplus Q_2 \\ \mathbf{P}_{\cdot:o} &= \mathbf{P}_1 \cup \mathbf{P}_2 \cup \{P_1 \wedge P_2 \mid P_1 \in \mathbf{P}_1 \wedge P_2 \in \mathbf{P}_2\} \\ \mathbf{X}_{\cdot:o} &= \mathbf{X}_1 \cup \mathbf{X}_2 \\ q_0 &\text{ the initial state of } \mathcal{A}_{\varphi_1}. \\ F_2 &\text{ the final states of } \mathcal{A}_{\varphi_2}. \end{aligned}$$

The transition relation $\Delta_{\cdot:o}$ is formally defined as:

$$\begin{aligned} \Delta_{\cdot:o} &= \Delta_1 \uplus \Delta_2 \\ &\uplus \{(q, \epsilon, (q, p_0, 0)) \mid q \in Q_1\} \\ &\uplus \{((q, p, b), P_1 \wedge P_2, L_1 \cup L_2, (q', p', 1)) \mid (q, P_1, L_1, q') \in \Delta_1 \wedge (p, P_2, L_2, p') \in \Delta_2 \wedge b \in \{0, 1\}\} \\ &\uplus \{((q, p, b), \epsilon, (q', p, b)) \mid (q, \epsilon, q') \in \Delta_1 \wedge p \in Q_2 \wedge b \in \{0, 1\}\} \\ &\uplus \{((q, p, b), \epsilon, (q, p', b)) \mid q \in Q_1 \wedge (p, \epsilon, p') \in \Delta_2 \wedge b \in \{0, 1\}\} \\ &\uplus \{((q, p, 1), \epsilon, p) \mid q \in F_1 \wedge p \in Q_2\} \end{aligned}$$

Cristian: Revisado hasta aqui.

Intuitively, given an stream S we capture the eventes given φ_1 , and at some point (the overlap) we start capturing the events for φ_2 too.

Next, we prove that the construction is correct, namely, $\llbracket \varphi_1 :o \varphi_2 \rrbracket(\bar{S}) = \llbracket \mathcal{A}_{\cdot:o} \rrbracket(\bar{S})$ for every stream $\bar{S}$. The proof is by double containment.

$\llbracket \varphi_1 :o \varphi_2 \rrbracket(\bar{S}) \subseteq \llbracket \mathcal{A}_{\cdot:o} \rrbracket(\bar{S})$. Let $C_1 \cup C_2 \in \llbracket \varphi_1 :o \varphi_2 \rrbracket(\bar{S})$ where $C_1 \in \llbracket \varphi_1 \rrbracket(\bar{S})$ and $C_2 \in \llbracket \varphi_2 \rrbracket(\bar{S})$. We know there exist accepting runs ρ_1 of $\mathcal{A}_{\varphi_1}$ and ρ_2 of $\mathcal{A}_{\varphi_2}$ over $\bar{S}$ such that $C_{\rho_1} = C_1$ and $C_{\rho_2} = C_2$. By the definition of a run, let:

$$\begin{aligned} \rho_1 &:= (q_0, i_1) \xrightarrow{t_1} (q_1, i_2) \xrightarrow{t_2} \dots \xrightarrow{t_n} (q_n, i_{n+1}) \\ \rho_2 &:= (p_0, j_1) \xrightarrow{t'_1} (p_1, j_2) \xrightarrow{t'_2} \dots \xrightarrow{t'_m} (p_m, j_{m+1}) \end{aligned}$$

where q_0 and p_0 are the initial states, $q_n \in F_1$, and $p_m \in F_2$. Note that by definition, $\text{start}(C_1) = i_1$, $\text{end}(C_1) = i_{n+1} - 1$, $\text{start}(C_2) = j_1$, and $\text{end}(C_2) = j_{m+1} - 1$.

Because $C_1 \cup C_2$ is an overlapping event, we know $i_1 \leq j_1 \leq i_{n+1} - 1 \leq j_{m+1} - 1$. To prove the containment, we construct the run $\rho_{\cdot:o}$ of $\mathcal{A}_{\cdot:o}$ by combining ρ_1 and ρ_2 . We define arbitrary indices $a \in [1..n]$ and $b \in [1..m]$ such that:

- a is the step in ρ_1 where the configuration first reaches stream position j_1 (i.e., $i_a = j_1$).
- b is the step in ρ_2 where the configuration first reaches stream position i_{n+1} (i.e., $j_b = i_{n+1}$).

140 We then move forward with the construction of $\rho_{:o}$, using these points when needed. First,
 141 the run $\rho_{:o}$ mimics ρ_1 from step 1 to $a - 1$ using transitions in Δ_1 :

$$142 \quad \rho_{:o} := (q_0, i_1) \xrightarrow{t_1} \dots \xrightarrow{t_{a-1}} (q_{a-1}, j_1) \dots$$

143 At this point we reach step a , here the stream is exactly at j_1 , which is where ρ_2 must begin.
 144 We use the ϵ -transition $(q_{a-1}, \epsilon, (q_{a-1}, p_0, 0)) \in \Delta_{:o}$:

$$145 \quad \rho_{:o} := \dots (q_{a-1}, j_1) \xrightarrow{\epsilon} ((q_{a-1}, p_0, 0), j_1) \dots$$

146 We now use both runs in the construction for the product state section of the transitions,
 147 carrying a bit b that is initially 0. We iterate over x and y where $x = a$ and $y = 1$. While
 148 $x \leq n$ and $y < b$, both runs are at the exact same stream position. At each step, we look at
 149 the transitions t_x and t'_y from ρ_1 and ρ_2 respectively:

- 150 ■ If $t_x = \epsilon$, we use $\{((q_{x-1}, p_{y-1}, b), \epsilon, (q_x, p_{y-1}, b))\} \subseteq \Delta_{:o}$. We advance $x \leftarrow x + 1$.
- 151 ■ If $t'_y = \epsilon$, we use $\{((q_{x-1}, p_{y-1}, b), \epsilon, (q_{x-1}, p_y, b))\} \subseteq \Delta_{:o}$. We advance $y \leftarrow y + 1$.
- 152 ■ If $t_x = (P, L)$ and $t'_y = (P', L')$, both runs are reading the current stream event. We
 153 use $\{((q_{x-1}, p_{y-1}, b), P \wedge P', L \cup L', (q_x, p_y, 1))\} \subseteq \Delta_{:o}$. We advance both $x \leftarrow x + 1$ and
 154 $y \leftarrow y + 1$, and the bit b becomes 1.

155 Because the overlap interval is valid ($j_1 \leq i_{n+1} - 1$), there is at least one stream position
 156 where both automata consume an event. Thus, the bit b is guaranteed to become 1 during
 157 this phase. By iterating these rules, the run reaches the point where ρ_1 is complete (we are
 158 in q_n) and ρ_2 is about to process step b (we are in p_{b-1}):

$$159 \quad \rho_{:o} := \dots ((q_{a-1}, p_0, 0), j_1) \xrightarrow{\text{iterative phase}} \dots \rightarrow ((q_n, p_{b-1}, 1), i_{n+1}) \dots$$

160 Since $q_n \in F_1$ and the bit is 1, we can use the ϵ -transition $((q_n, p_{b-1}, 1), \epsilon, p_{b-1}) \in \Delta_{:o}$:

$$161 \quad \rho_{:o} := \dots ((q_n, p_{b-1}, 1), i_{n+1}) \xrightarrow{\epsilon} (p_{b-1}, i_{n+1}) \dots$$

162 Finally, the run follows the remaining transitions of ρ_2 from step b to m using the transitions
 163 in Δ_2 :

$$164 \quad \rho_{:o} := \dots (p_{b-1}, i_{n+1}) \xrightarrow{t'_b} \dots \xrightarrow{t'_m} (p_m, j_{m+1})$$

165 Because $p_m \in F_2$, the run $\rho_{:o}$ is accepting. Furthermore, by the construction of the iterative
 166 phase, every variable mapping L captured by ρ_1 and L' captured by ρ_2 are explicitly merged
 167 via $L \cup L'$. Thus, $C_{\rho_{:o}} = C_1 \cup C_2$. Therefore, $\llbracket \varphi_1 :o \varphi_2 \rrbracket(\bar{S}) \subseteq \llbracket \mathcal{A}_{:o} \rrbracket(\bar{S})$.

168 $\llbracket \mathcal{A}_{:o} \rrbracket(\bar{S}) \subseteq \llbracket \varphi_1 :o \varphi_2 \rrbracket(\bar{S})$. Let $C \in \llbracket \mathcal{A}_{:o} \rrbracket(\bar{S})$. By definition, there exists an accepting run
 169 $\rho_{:o}$ in $\mathcal{A}_{:o}$ over $\bar{S}$ such that $C_{\rho_{:o}} = C$. Because $\rho_{:o}$ starts at q_0 and ends at an accepting
 170 state $p_m \in F_2$, it must, by the construction of $\Delta_{:o}$, contain exactly one entry ϵ -transition
 171 $(q, \epsilon, (q, p_0, 0))$ and exactly one exit ϵ -transition $((q', p, 1), \epsilon, p')$. Therefore, the run is strictly
 172 partitioned into three sections: one in Q_1 , an overlap phase in $Q_1 \times Q_2 \times \{0, 1\}$, and one in
 173 Q_2 . Lets call them *prefix*, *overlap*, and *suffix*, respectively.

174 We then can separate $\rho_{:o}$ into two separate runs, ρ_1 and ρ_2 , by isolating the components
 175 from the first and last partition.

- 176 ■ **Construction of ρ_1 :** For every state in the prefix and overlap phases of $\rho_{:o}$, we take the
 177 first coordinate (Q_1). Because every transition in $\Delta_{:o}$ that affects the first coordinate is
 178 strictly derived from Δ_1 , ρ_1 is a perfectly valid run in $\mathcal{A}_{\varphi_1}$. Furthermore, because the
 179 overlap phase ends by taking $((q', p, 1), \epsilon, p')$, where $q' \in F_1$, ρ_1 is an accepting run. Let
 180 C_1 be the complex event generated by ρ_1 .

181 ■ **Construction of ρ_2 :** For every state in the overlap and suffix phases of $\rho_{:o}$, we take the
 182 second coordinate (Q_2). Similarly, because the second coordinate is only modified by
 183 rules derived from Δ_2 , ρ_2 is a valid run in $\mathcal{A}_{\varphi_2}$. Because $\rho_{:o}$ ultimately ends in F_2 , ρ_2 is
 184 an accepting run. Let C_2 be the complex event generated by ρ_2 .

185 By induction, since ρ_1 and ρ_2 are accepting runs of $\mathcal{A}_{\varphi_1}$ and $\mathcal{A}_{\varphi_2}$, respectively, we have
 186 $C_1 \in \llbracket \varphi_1 \rrbracket(\bar{S})$ and $C_2 \in \llbracket \varphi_2 \rrbracket(\bar{S})$.

187 Crucially, the exit transition out of the overlap phase is strictly conditioned on the state
 188 being $((q', p, 1), \epsilon, p')$. By the construction of $\Delta_{:o}$, the only way the bit transitions from 0
 189 to 1 is if $\rho_{:o}$ executes at least one synchronized reading transition involving $P_1 \wedge P_2$ during
 190 the overlap phase. This strictly guarantees that there is at least one stream position k
 191 where both ρ_1 and ρ_2 consume an event simultaneously. Consequently, $k \in \text{interval}(C_1)$ and
 192 $k \in \text{interval}(C_2)$. This implies that $\text{start}(C_1) \leq k \leq \text{end}(C_1)$ and $\text{start}(C_2) \leq k \leq \text{end}(C_2)$,
 193 from which it naturally follows that $\text{start}(C_1) \leq \text{start}(C_2) \leq \text{end}(C_1) \leq \text{end}(C_2)$. This directly
 194 resolves the interval constraints without ambiguity.

195 Because the overlapped transitions in $\Delta_{:o}$ explicitly construct the variable mapping via
 196 $L_1 \cup L_2$, it follows that $C = C_1 \cup C_2$. Therefore, $C \in \llbracket \varphi_1 :o \varphi_2 \rrbracket(\bar{S})$. ◀

197 — References —

- 198 1 Marco Buchi, Alejandro Grez, Andrés Quintana, Cristian Riveros, and Stijn Vansummeren.
 199 CORE: a complex event recognition engine. *VLDB*, 15(9):1951–1964, 2022.
- 200 2 Alejandro Grez, Cristian Riveros, Martín Ugarte, and Stijn Vansummeren. A formal framework
 201 for complex event recognition. *ACM Trans. Database Syst.*, 46(4):16:1–16:49, 2021. doi:
 202 10.1145/3485463.